





Aula 04:

Autenticação na API com JWT

JSON Web Token

A autenticação de uma API com JSON Web Token (JWT) é um método usado para verificar a identidade de usuários e proteger o acesso a sistemas.

JWT é um padrão aberto que define uma maneira compacta e segura de transmitir informações entre partes como um objeto JSON.



Para instalá-lo na aplicação rode o comando:

npm install jsonwebtoken

JSON Web Token

Login: O cliente (usuário) envia suas credenciais (como nome de usuário e senha) para a API.

Geração do Token: Se as credenciais forem válidas, o servidor gera um token JWT assinado com uma chave secreta ou com um par de chaves (pública/privada). Esse token contém informações do usuário, como um identificador (ID), permissões, e uma data de expiração.

Envio do Token: O token é enviado de volta ao cliente e armazenado, geralmente em um cookie seguro ou local storage.

Uso do Token: Para cada solicitação futura à API, o cliente inclui o token no cabeçalho da requisição (normalmente no campo Authorization com o esquema Bearer).

Validação do Token: O servidor valida o token recebido, verificando sua assinatura e outros dados (como validade) antes de permitir o acesso aos recursos da API.





Aula 04.1:

Cadastro de usuário e login

Criando o Model de Usuário

Criaremos o modelo que irá representar a entidade **User** em nosso sistema. Para isso, dentro da pasta Models, crie o arquivo **User.js**. Esse arquivo terá a seguinte estrutura:




```
import mongoose from 'mongoose'

const userSchema = new mongoose.Schema({
    name: String,
    email: String,
    password: String,
})

const User = mongoose.model("User",
userSchema)

export default User
```

1 - Importação da biblioteca Mongoose;

2 – Criação de um novo **Schema***

3 – Criação do model. Aqui informamos que deve ser criado uma coleção que receberá o nome de users quando for para o banco de dados.

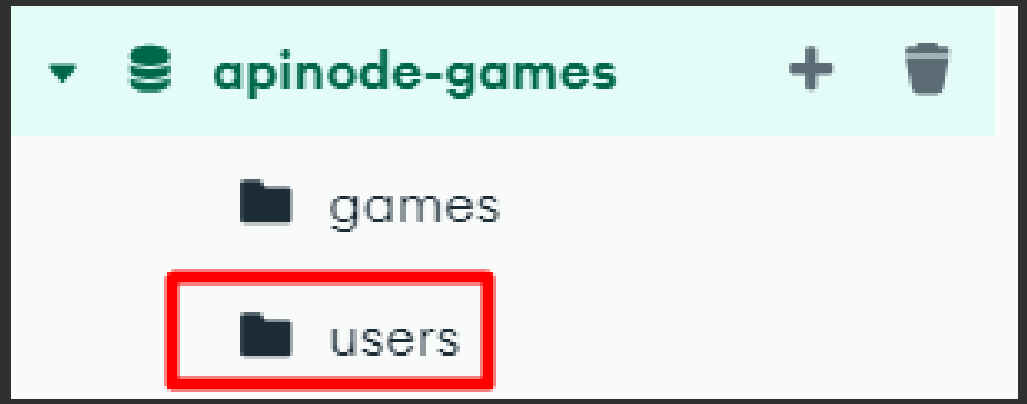
4 - Eportação do módulo.

Criando o Model de Usuário

Após isso, realize a importação do model no arquivo **index.js**

```
import User from './models/Users.js'
```

Em seguida, rode a aplicação e verifique se a coleção **users** foi criada no banco de dados.



Cadastrando usuários

Crie o arquivo **userService.js**. Para começarmos a cadastrar usuários na API, criaremos um método chamado **Create()** na classe **userService** que irá inserir novos usuários no banco de dados.

```
1  import User from "../models/Users.js";
2
3  class userService {
4    async Create(name, email, password) {
5      try {
6        const newUser = new User({
7          name,
8          email,
9          password,
10       });
11       await newUser.save();
12     } catch (error) {
13       console.log(error);
14     }
15   }
16 }
17
18 export default new userService();
```

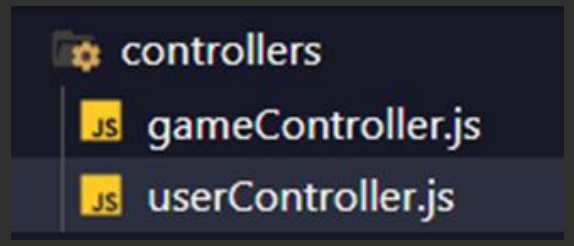


Cadastrando usuários

Após isso, crie o arquivo **userController.js**, nele criaremos a constante **createUser** que recebe uma função assíncrona.

Nessa função será coletado os campos vindos do corpo da requisição POST, em seguida será chamado o método **Create()** do service para cadastrar o usuário no banco.

Lembre-se também de **exportar createUser** no final do arquivo.

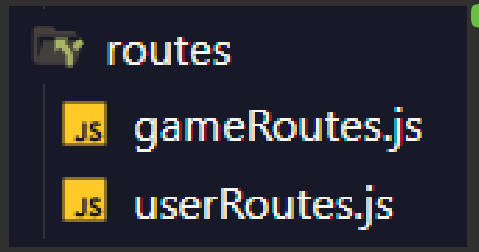



```
1  import userService from "../services/userService.js";
2  import jwt from "jsonwebtoken";
3  const JWTSecret = "apigamessecret";
4
5  //Cadastrando um Usuário
6  const createUser = async (req, res) => {
7    try {
8      const { name, email, password } = req.body;
9      await userService.Create(name, email, password);
10     res.sendStatus(201); //Código 201 (CREATED)
11   } catch (error) {
12     console.log(error);
13     res.sendStatus(500); // Erro interno do servidor
14   }
15 };
16
17 export default { createUser };
```

Cadastrando usuários

Agora devemos apenas criar um novo **endpoint** "**/user**" no arquivo **userRoutes.js**, que receberá a requisição **POST** e chamará o método **createUser()** do userController para tratar a requisição:

```
1  import express from 'express'
2  const userRoutes = express.Router()
3  import userController from '../controllers/userController.js'
4
5  // Endpoint para cadastrar um Usuário
6  userRoutes.post("/user", userController.createUser)
7
8  export default userRoutes
```



Feito isso, importe as rotas do **userRoutes** no arquivo principal **index.js**, assim como feito com gameRoutes. Agora, testaremos a requisição no **Insomnia** ou **Postman**, enviando uma requisição **POST** para a rota "**/user**". Será necessário enviar os dados que se deseja cadastrar no corpo da requisição (**BODY**) em formato **JSON**. Envie os campos "**email**" e "**password**" em um JSON. Confira se o usuário é cadastrado.

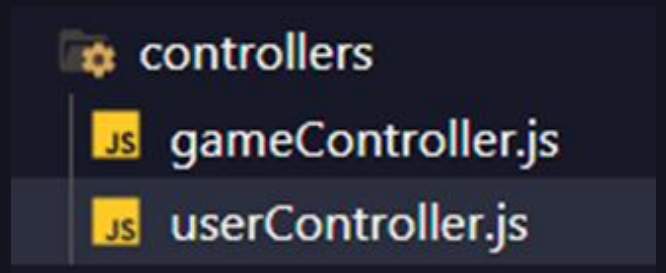
Gerando o Token de autenticação

No arquivo **userService.js**. Criaremos mais um método chamado **getOne()** na classe **userService** que irá retornar os dados de um único usuário, quando for realizado o login.

```
1    async getOne(email) {
2      try {
3        const user = await User.findOne({ email: email });
4        return user;
5      } catch (error) {
6        console.log(error);
7      }
8    }
9  }
10
11 export default new userService();
```



Nos próximos slides terá o código do método **loginUser()** para ser inserido no arquivo **userController.js**. Esse método será responsável por fazer o login do usuário e devolver um token de autenticação JWT.




```
// Função para LOGIN do Usuário
const loginUser = async (req, res) => {
  try {
    const { email, password } = req.body;
    // E-mail válido
    if (email != undefined) {
      const user = await userService.getOne(email);
      // Usuário encontrado
      if (user != undefined) {
        // Senha correta
        if (user.password == password) {
          jwt.sign(
            { id: user._id, email: user.email },
            JWTSecret,
            { expiresIn: "48h" },
            (err, token) => {
              if (err) {
                res.status(400); // Bad request
                res.json({ err: "Falha interna" });
              } else {
                res.status(200); // OK
                res.json({ token: token });
              }
            }
          );
          // Senha incorreta
```

```
20                } else {
21                  res.status(200); // OK
22                  res.json({ token: token });
23                }
24              }
25            );
26            // Senha incorreta
27          } else {
28            res.status(401); // Unauthorized
29            res.json({ err: "Credenciais inválidas!" })
30          }
31          // Usuário não encontrado
32        } else {
33          res.status(404); // Not Found
34          res.json({ err: "O e-mail enviado não foi enc
35        }
36        // E-mail inválido
37      } else {
38        res.status(400); // Bad request
39        res.json({ err: "O e-mail enviado é inválido."
40      }
41    } catch (error) {
42      console.log(error);
43      res.sendStatus(500); // Internal Server Error
44    }
45  };
46
47  export default { createUser, loginUser };
```

Gerando o Token de autenticação

Agora devemos apenas criar um novo endpoint "**/auth**"no arquivo **userRoutes.js**, que receberá a requisição **POST** e chamará o método **loginUser()** do userController para tratar a requisição de login:

```
1  // Endpoint para Login do Usuário
2  userRoutes.post("/auth", userController.loginUser)
```

Agora, testaremos a requisição no **Insomnia** ou **Postman** enviando uma requisição **POST** para a rota "**/auth**". Envie os campos "email" e "password" em um JSON. A API deve retornar o **token de autenticação**.





Aula 04.2:

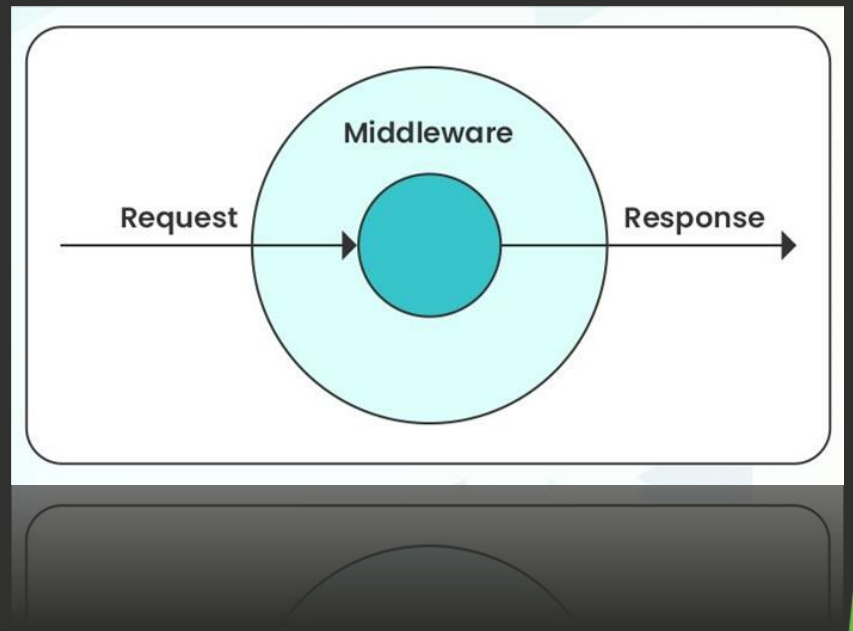
Middleware de autenticação

Middleware de autenticação

um middleware é uma função que tem acesso ao objeto de requisição (**req**), ao objeto de resposta (**res**), e à próxima função de middleware no ciclo de solicitação/resposta de uma aplicação.

O middleware pode executar várias tarefas, como:

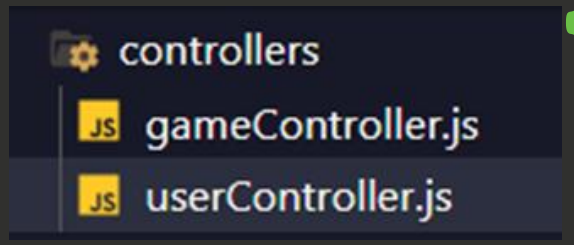
- Executar qualquer código.
- Modificar o objeto de requisição (req) ou resposta (res).
- Encerrar o ciclo de requisição/resposta, enviando uma resposta ao cliente.
- Chamar a próxima função de middleware no stack usando a função **next()**. Se o middleware atual não terminar o ciclo de requisição/resposta, ele deve invocar next() para passar o controle para o próximo middleware.



Middleware de autenticação

O primeiro passo é exportar o **JWTSecret** no arquivo **userController.js** para podermos usá-lo no middleware de autenticação.

```
1  export default { createUser, loginUser, JWTSecret }
```



Após isso, criaremos nosso middleware de autenticação. O middleware irá verificar se o cliente enviou um **tokenJWT** (Json Web Token) no cabeçalho da requisição, sob o campo "authorization". Se o token estiver presente e for válido, ele é decodificado, e as informações do usuário (como ID e e-mail) são anexadas ao objeto req. Isso permite que essas informações sejam acessadas nas próximas etapas da requisição. Caso o token seja inválido ou ausente, a requisição é rejeitada com um status **401** (não autorizado), e uma mensagem de erro é enviada ao cliente. Para isso, crie uma pasta com o nome **middleware** na raiz do projeto e em seguida um arquivo dentro da pasta chamado **Auth.js**. O código deste arquivo está nos próximos slides.

middleware

Auth.js

```js
import jwt from "jsonwebtoken";
import userController from "../controllers/userController.js";

// Função de Autenticação com JWT - Json Web Token
const Authorization = (req, res, next) => {
  const authToken = req.headers["authorization"];
  if (authToken != undefined) {
    const bearer = authToken.split(" ");
    var token = bearer[1];
    jwt.verify(token, userController.JWTSecret, (err, data) => {
      if (err) {
        res.status(401);
        res.json({ err: "Token inválido!" });
      } else {
        req.token = token;
        req.loggedUser = {
          id: data.id,
          email: data.email,
        };
        next();
      }
    });
  } else {
    res.status(401);
    res.json({ err: "Token inválido" });
  }
};

export default { Authorization };
```

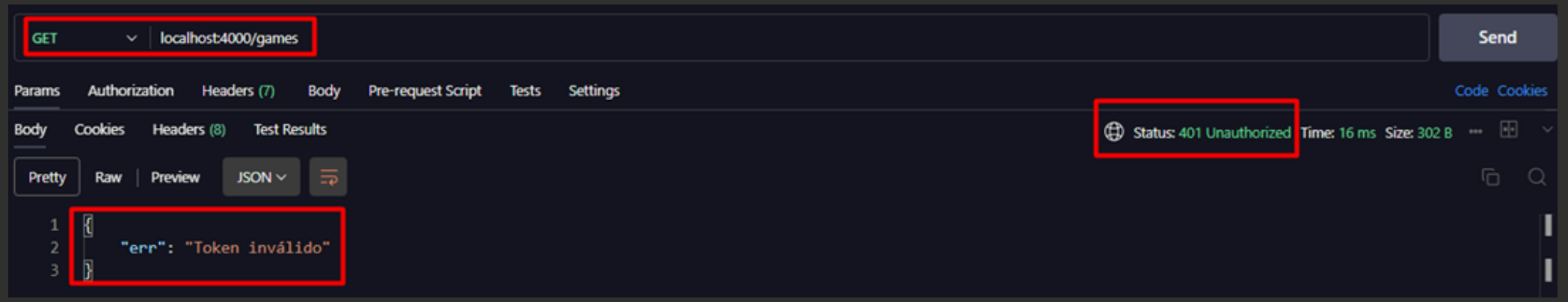
Middleware de autenticação

Após isso, devemos importar o middleware de autenticação no arquivo **gameRoutes.js**, e adicioná-lo em todas as rotas, conforme mostra a imagem abaixo:

```
1  import express from 'express'
2  const gameRoutes = express.Router()
3  import gameController from '../controllers/gameController.js'
4  import Auth from '../middleware/Auth.js'
5
6  // Endpoint para listar todos os Games
7  gameRoutes.get("/games", Auth.Authorization, gameController.getAllGames)
8
9  // Endpoint para cadastrar um Game
10 gameRoutes.post("/game", Auth.Authorization, gameController.createGame)
11
12 // Endpoint para deletar um Game
13 gameRoutes.delete("/game/:id", Auth.Authorization, gameController.deleteGame)
14
15 // Endpoint para alterar um Game
16 gameRoutes.put("/game/:id", Auth.Authorization, gameController.updateGame)
17
18 // Endpoint para listar um único Game
19 gameRoutes.get("/game/:id", Auth.Authorization, gameController.getOneGame)
20
21 export default gameRoutes
```

Middleware de autenticação

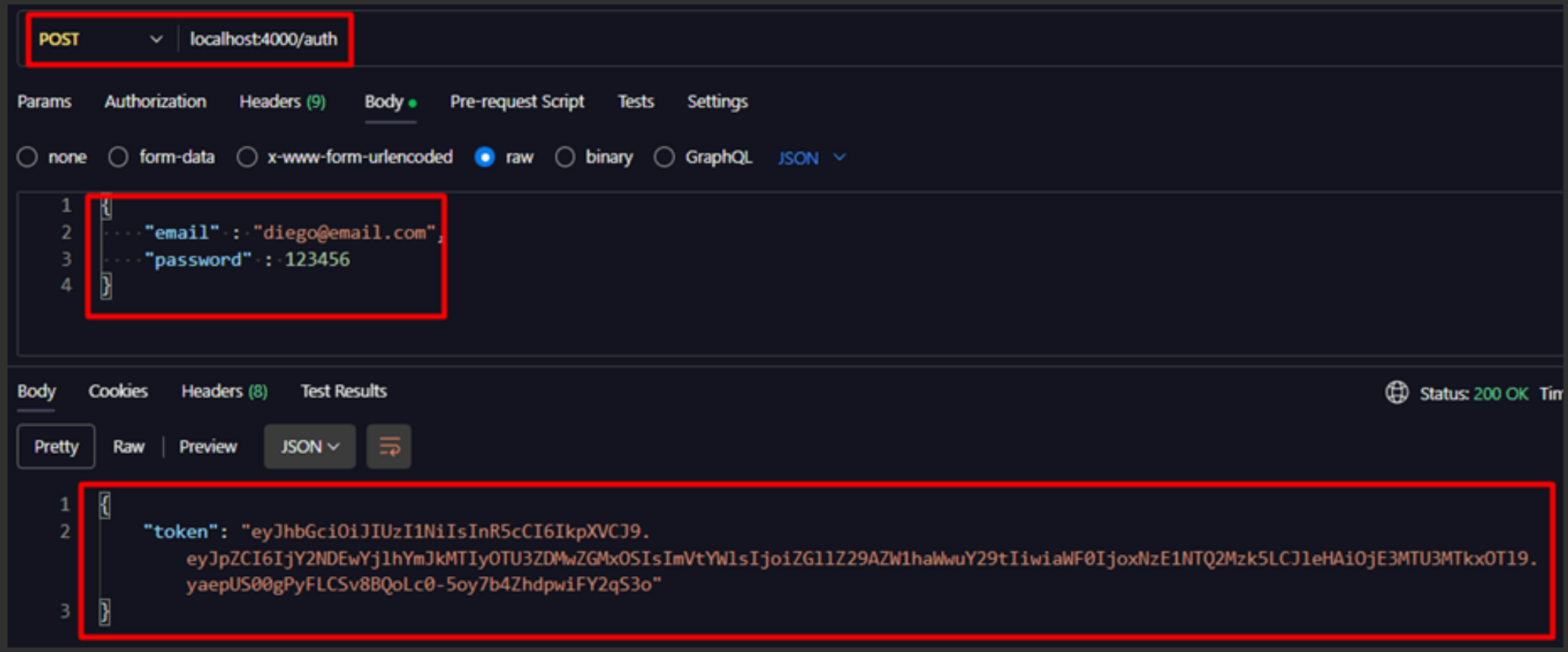
Por fim, realizaremos os testes no **Insomnia** ou **Postman**. No exemplo abaixo será usado o Postman. Agora, ao tentarmos acessar a rota "**/games**" é necessário informar o token de autenticação, caso contrário a API retornará o código de status **401** (**Não autorizado**).



Agora para ter acesso a listagem de games precisamos primeiramente realizar o login enviando **email** e **password** para a rota "**/auth**".

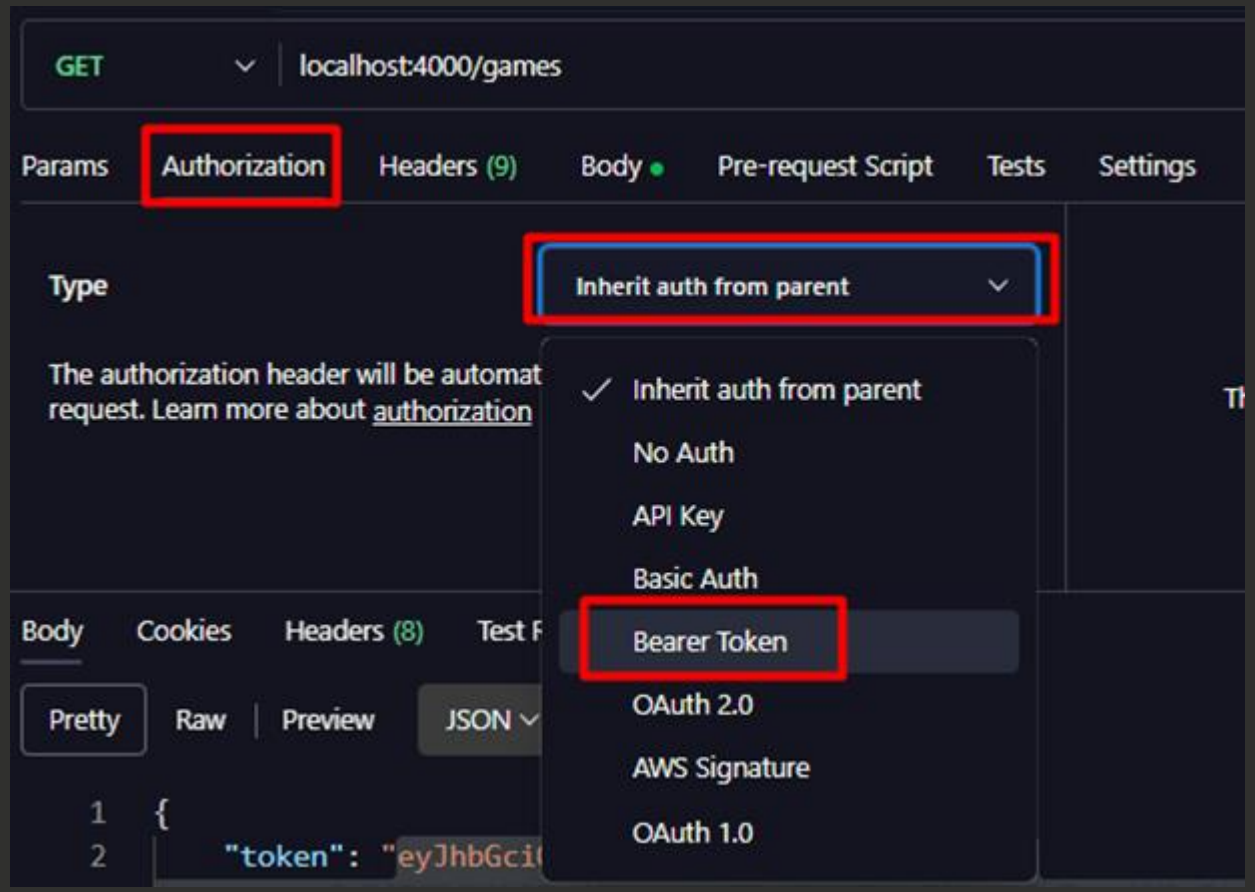
Middleware de autenticação

Ao realizarmos o login, a API nos retorna o token de autenticação:



Middleware de autenticação

Agora devemos inserir o token no cabeçalho da requisição, informando o tipo do token que é o **Bearer Token**. Segue logo abaixo um exemplo de como fazer isso no Postman.



Middleware de autenticação

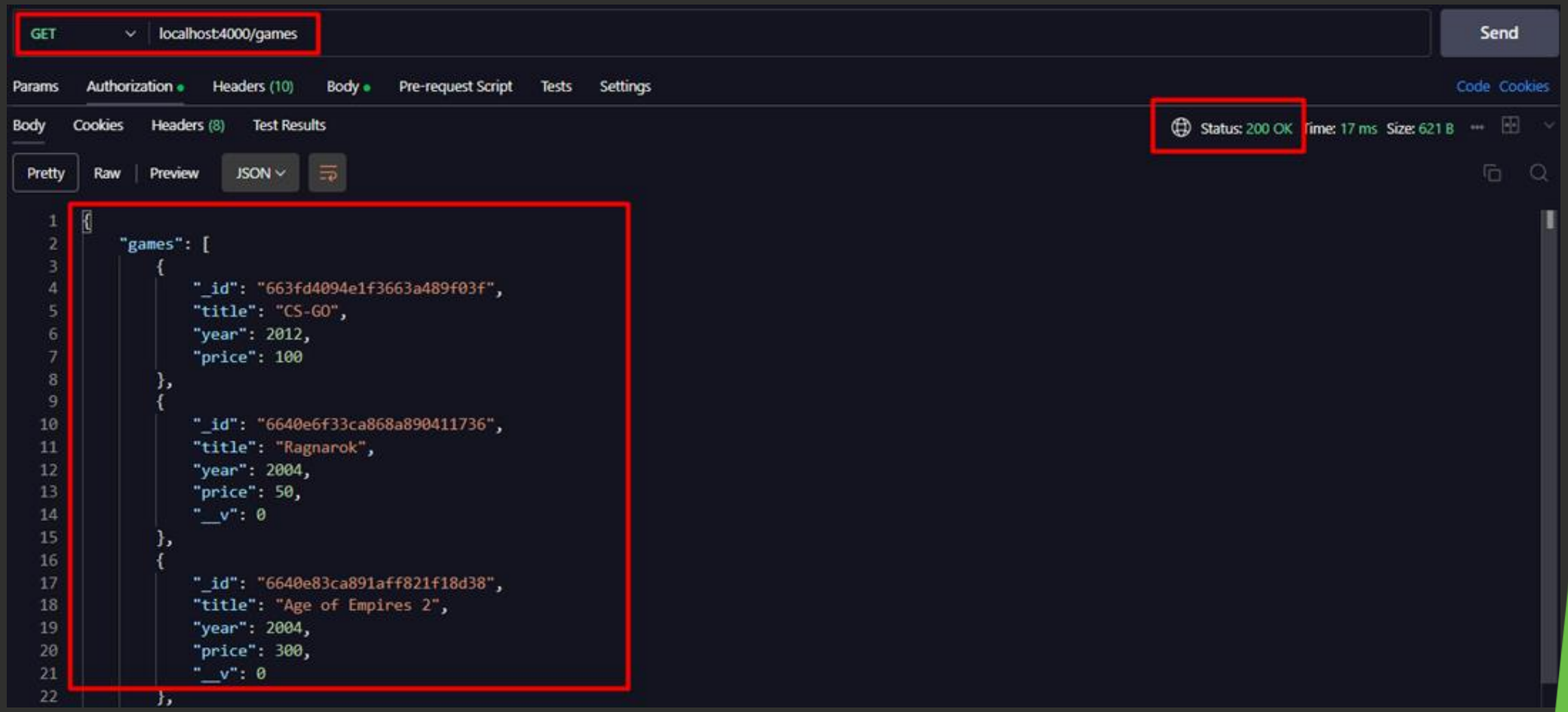
Selecionado o tipo do token cole no espaço indicado o token que foi retornado pela API no momento do login:



Após isso, tente realizar uma nova requisição para a rota "**/games**" para verificar se o acesso será concedido.

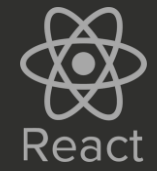
Middleware de autenticação

Com a devida autenticação, agora a API nos permite acessar o recurso normalmente e temos acesso a lista de games. Isso serve também para as outras requisições da API.













Aula 04:

Autenticação na API com JWT